Security Review Report
NM-0585 Daimo Universal Address

NETHERMIND SECURITY

(July 31, 2025)

Contents

1 Executive Summary **2**

2 Audited Files **3**

3 Summary of Issues **3**

4 System Overview **3**
4.1 Core Components . . . 3
4.2 Contract Interaction Diagram . . . 4

5 Risk Rating Methodology **4**

6 Issues **5**
6.1 [Low] Malicious relayer can cause user to lose `MIN_START_TOKEN_OUT - 1` tokens . . . 5
6.2 [Low] Stablecoin UA intents can fail and cannot be refunded . . . 6
6.3 [Best Practices] `BridgeReceiver` immutable `ua` has misleading name . . . 6

7 Documentation Evaluation **7**

8 Test Suite Evaluation **8**
8.1 Compilation Output . . . 8
8.2 Tests Output . . . 8
8.2.1 Foundry Tests Output . . . 8

9 About Nethermind **10**

1 Executive Summary

This document presents the results of a security review conducted by Nethermind Security for Daimo-pay. The **Universal Address** protocol is a decentralized system designed for secure, deterministic, and trustless cross-chain transfers of stablecoins across Ethereum Virtual Machine (EVM)-compatible blockchains. The protocol utilizes deterministic contract addresses generated via the `CREATE2` opcode, ensuring reliable and predictable asset custody and transfers.

The operational flow of the Universal Address protocol clearly defines procedures for same-chain and cross-chain stablecoin transfers. Users initiate transfers by depositing assets into deterministic vaults, after which `relayers` coordinate the bridging and settlement processes through the Universal Address Manager. The protocol facilitates optional token swaps, allows `relayers` to expedite asset delivery via fast-finalization, and ensures reliable settlement at predictable receiver addresses. Robust exception handling is also incorporated, permitting permissionless refunds and completion of transfers, thereby ensuring operational resilience and reliability.

The audit comprises 1198 lines of Solidity code. The audit was performed using (a) manual analysis of the codebase, (b) automated analysis tools, and (c) creation of test cases.

Along this document, we report 3 points of attention, where: two are classified as `Low` and one is classified as `Best Practices` severity. The issues are summarized in Fig. 1.

This document is organized as follows. Section 2 presents the files in the scope. Section 3 summarizes the issues. Section 4 presents the system overview. Section 5 discusses the risk rating methodology. Section 6 details the issues. Section 7 discusses the documentation provided by the client for this audit. Section 8 presents the test suite evaluation and automated tools used. Section 9 concludes the document.

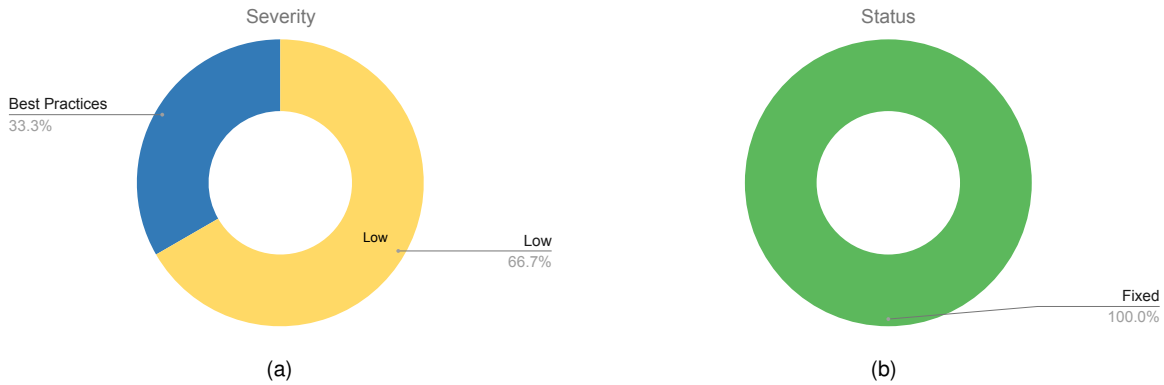


Fig. 1: Distribution of issues: Critical (0), **High** (0), **Medium** (0), **Low** (2), **Undetermined** (0), **Informational** (0), **Best Practices** (1). **Distribution of status: Fixed** (3), **Acknowledged** (0), **Mitigated** (0), **Unresolved** (0)

Summary of the Audit

Audit Type	Security Review
Initial Report	July 21, 2025
Final Report	July 31, 2025
Repository	Daimo
Initial Commit	`ef97c77fe68f8ff12a98087f2f74e2cd12fc39c1`
Final Commit	`80312d47e98a21b7daa4078d89ffe0e43bf8b1eb`
Documentation	Universal Address Audit
Documentation Assessment	Medium
Test Suite Assessment	Medium

2 Audited Files

	Contract	LoC	Comments	Ratio	Blank	Total
1	packages/contract/src/UniversalAddressBridger.sol	86	25	29.1%	18	129
2	packages/contract/src/UniversalAddressFactory.sol	56	15	26.8%	7	78
3	packages/contract/src/UniversalAddressManager.sol	407	186	45.7%	75	668
4	packages/contract/src/SharedConfig.sol	44	51	115.9%	23	118
5	packages/contract/src/UniversalAddress.sol	50	30	60.0%	14	94
6	packages/contract/src/TokenUtils.sol	87	14	16.1%	10	111
	Total	**730**	**321**	**44.0%**	**147**	**1198**

3 Summary of Issues

	Finding	Severity	Update
1	Malicious relayer can cause user to lose `MIN_START_TOKEN_OUT - 1` tokens	Low	Fixed
2	Stablecoin UA intents can fail and cannot be refunded	Low	Fixed
3	`BridgeReceiver` immutable `ua` has misleading name	Best Practices	Fixed

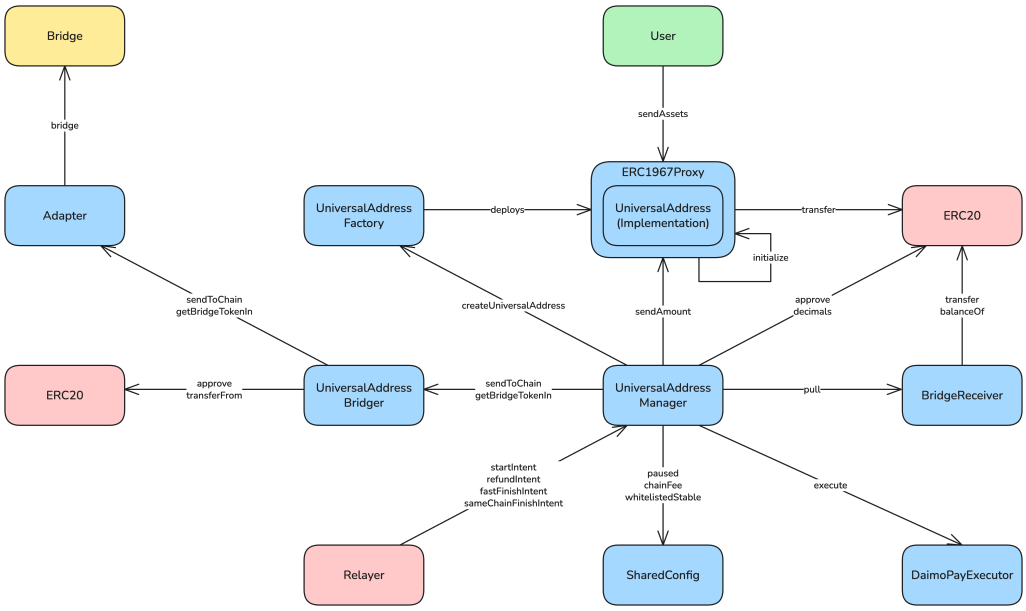
4 System Overview

The Universal Address (UA) protocol implements trustless, deterministic, and secure cross-chain stablecoin transfers across Ethereum Virtual Machine (EVM)-compatible chains. The system leverages the `CREATE2 opcode` for deterministic contract addresses, ensuring predictability and reliability of fund transfers and storage.

4.1 Core Components

- Universal Address Factory
  - Deploys deterministic (Universal Address) UA vault contracts using the `CREATE2` opcode.
  - Creates minimal proxies (`ERC1967Proxy`) pointing to a singleton implementation of `UniversalAddress`.
- Universal Address Manager
  - Central orchestrator managing the lifecycle of cross-chain stablecoin transfers.
  - Handles initiation (`startIntent`), fast-finalization (`fastFinishIntent`), and settlement (`claimIntent`) of transfers.
  - Facilitates same-chain payments and refunds unsupported assets.
  - Utilizes a deterministic receiver contract (`BridgeReceiver`) to manage bridged funds.
- Universal Address Bridger
  - Multiplexes between bridge-specific adapters (`IDaimoPayBridger`), abstracting bridging logic.
  - Ensures the bridging token aligns with the configured destination token.
- Universal Address
  - Stateless minimal vault deployed deterministically, securing assets per specified transfer routes.
  - Controlled exclusively by an authorized escrow (Universal Address Manager).
- Shared Config
  - Centralized, upgradeable contract managing global configurations such as stablecoin whitelisting, global pauses, and numeric safeguards.
  - Allows governance-controlled updates via the UUPS upgrade pattern.

4.2 Contract Interaction Diagram



5 Risk Rating Methodology

The risk rating methodology used by Nethermind Security follows the principles established by the OWASP Foundation. The severity of each finding is determined by two factors: **Likelihood** and **Impact**.

Likelihood measures how likely the finding is to be uncovered and exploited by an attacker. This factor will be one of the following values:

a) **High**: The issue is trivial to exploit and has no specific conditions that need to be met;

b) **Medium**: The issue is moderately complex and may have some conditions that need to be met;

c) **Low**: The issue is very complex and requires very specific conditions to be met.

When defining the likelihood of a finding, other factors are also considered. These can include but are not limited to motive, opportunity, exploit accessibility, ease of discovery, and ease of exploit.

Impact is a measure of the damage that may be caused if an attacker exploits the finding. This factor will be one of the following values:

a) **High**: The issue can cause significant damage, such as loss of funds or the protocol entering an unrecoverable state;

b) **Medium**: The issue can cause moderate damage, such as impacts that only affect a small group of users or only a particular part of the protocol;

c) **Low**: The issue can cause little to no damage, such as bugs that are easily recoverable or cause unexpected interactions that cause minor inconveniences.

When defining the impact of a finding, other factors are also considered. These can include but are not limited to Data/state integrity, loss of availability, financial loss, and reputation damage. After defining the likelihood and impact of an issue, the severity can be determined according to the table below.

		Severity Risk		
Impact	**High**	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Info/Best Practices	Low	Medium
	Undetermined	Undetermined	Undetermined	Undetermined
		Low	**Medium**	**High**
		Likelihood		

To address issues that do not fit a High/Medium/Low severity, Nethermind Security also uses three more finding severities: **Informational**, **Best Practices**, and **Undetermined**.

a) **Informational** findings do not pose any risk to the application, but they carry some information that the audit team intends to pass to the client formally;

b) **Best Practice** findings are used when some piece of code does not conform with smart contract development best practices;

c) **Undetermined** findings are used when we cannot predict the impact or likelihood of the issue.

6 Issues

6.1 [Low] Malicious relayer can cause user to lose MIN_START_TOKEN_OUT - 1 tokens

File(s): packages/contract/src/UniversalAddressManager.sol

Description: Once a user has funded a UA intent, a relayer calls startIntent on their behalf to complete necessary swaps and send the funds to the destination chain via the UniversalAddressBridger contract. Part of the validation during this function is a check to ensure that the amount of bridgeTokenOut from the bridge will meet the minimum requirements in the SharedConfig contract num function using key MIN_START_TOKEN_OUT_KEY.

```
function startIntent(...) external nonReentrant notPaused {
    // ...
    uint256 outAmount = bridgeTokenOut.amount;
    require(
        outAmount >= cfg.num(MIN_START_TOKEN_OUT_KEY),
        "UAM: amount < min"
    );
}
```

The UniversalAddressManager allows relayers to split the funds from one UA address into multiple intents. This feature is necessary to allow for UA addresses with significant funds, where one single bridge may not have enough liquidity, instead allowing the funds to be broken into smaller parts and transferred over multiple bridges. A malicious relayer could abuse this behavior to send an amount that leaves the remaining funds to be exactly under the limit of MIN_START_TOKEN_OUT, which would leave those funds stuck on the UA address.

Consider the following example: A user attempts to transfer 10 USDC from chain A to chain B, and the MIN_START_TOKEN_OUT is set to 5 USDC. A malicious relayer calls startIntent and transfers 5.01 USDC to the destination chain, leaving the remaining 4.99 USDC in the user's UA address. The 4.99 USDC in the UA will remain and cannot be transferred, as this is under the minimum requirement. In this worst-case scenario, the user would lose 50% of their deposited amount.

Recommendation(s): The MIN_START_TOKEN_OUT will be set to 1 USD according to discussions with the Daimo team. A mechanism could be introduced that ensures the entire amount is transferred as long as some upper threshold is not met, or the implementation could be left as is, with 1 USD losses being deemed acceptable.

Status: Fixed

Update from the client: Fixed in commit 80312d47e98a21b7daa4078d89ffe0e43bf8b1eb. We've removed the MAX_START_TOKEN_OUT config. Now, the PARTIAL_START_THRESHOLD_KEY serves as the threshold that determines when the relayer is allowed to partially start an order.

6.2 [Low] Stablecoin UA intents can fail and cannot be refunded

File(s): packages/contract/src/UniversalAddressManager.sol

Description: The UniversalAddressManager allows an intent to be refunded when a user funds a UA with non-whitelisted assets, as these are not supported, but there is no mechanism that supports refunds for whitelisted assets. This design approach implies that any UA intent using whitelisted assets should not need a refund and funds will always reach their intended target. However, it is possible for a user to set their UniversalAddressRoute fields in such a way that even a UA intent with whitelisted funds can fail, particularly toChainId and toToken.

When a relayer calls startIntent on behalf of a user, the funds must be bridged to the destination chain using the UniversalAddressBridger. During startIntent the UAB function getBridgeTokenIn is called, which finds the appropriate adapter and determines the correct input token for the selected bridge. If an unsupported toChainId or bridgeTokenOut is provided, then execution will revert, as shown below:

```
function _getAdapter(...) ... {
    // ...

    // The specified destination chain must be supported
    adapter = chainIdToBridger[toChainId];
    require(address(adapter) != address(0), "UAB: unknown chain");

    // The specified token out must be supported otherwise revert
    address tokOut = chainIdToStableOut[toChainId];
    require(address(bridgeTokenOut.token) == tokOut, "UAB: token mismatch");
    // ...
}
```

Since execution fails during startIntent, funds will never be taken out of the user's UA address, this is a case that would require a refund to recover the funds, but the refundIntent function only supports non-whitelisted tokens. Due to the whitelisted stable check in the refund function it will not be possible to recover the user's funds.

Recommendation(s): Consider introducing a mechanism to allow for whitelisted tokens to be refunded in the case that startIntent execution fails due to a misconfigured UniversalAddressRoute.

Status: Fixed

Update from the client: Fixed in commit b40d727eeacf0a9a353ad2986e22179e2f0f186a. A refund mechanism for UA intents with invalid destination chain IDs has been added.

6.3 [Best Practices] BridgeReceiver immutable ua has misleading name

File(s): packages/contract/src/UniversalAddressManager.sol

Description: The BridgeReceiver contract has an immutable named ua which tracks the address from which it was deployed. The name ua implies that the address this variable tracks is a UniversalAddress, however, this contract will actually be deployed by the UniversalAddressManager.

```
contract BridgeReceiver {
    // ...
    address payable public immutable ua;

    constructor() {
        ua = payable(msg.sender);
    }
    // ...
}
```

Recommendation(s): Consider changing the name of the variable ua to uam to more accurately reflect what is stored in the variable.

Status: Fixed

Update from the client: Fixed in commit 4bf78a6545f87ad762e64f6f33f667699f89bc14. The variable name has been updated.

7 Documentation Evaluation

Software documentation refers to the written or visual information that describes the functionality, architecture, design, and implementation of software. It provides a comprehensive overview of the software system and helps users, developers, and stakeholders understand how the software works, how to use it, and how to maintain it. Software documentation can take different forms, such as user manuals, system manuals, technical specifications, requirements documents, design documents, and code comments. Software documentation is critical in software development, enabling effective communication between developers, testers, users, and other stakeholders. It helps to ensure that everyone involved in the development process has a shared understanding of the software system and its functionality. Moreover, software documentation can improve software maintenance by providing a clear and complete understanding of the software system, making it easier for developers to maintain, modify, and update the software over time. Smart contracts can use various types of software documentation. Some of the most common types include:

- Technical whitepaper: A technical whitepaper is a comprehensive document describing the smart contract's design and technical details. It includes information about the purpose of the contract, its architecture, its components, and how they interact with each other;
- User manual: A user manual is a document that provides information about how to use the smart contract. It includes step-by-step instructions on how to perform various tasks and explains the different features and functionalities of the contract;
- Code documentation: Code documentation is a document that provides details about the code of the smart contract. It includes information about the functions, variables, and classes used in the code, as well as explanations of how they work;
- API documentation: API documentation is a document that provides information about the API (Application Programming Interface) of the smart contract. It includes details about the methods, parameters, and responses that can be used to interact with the contract;
- Testing documentation: Testing documentation is a document that provides information about how the smart contract was tested. It includes details about the test cases that were used, the results of the tests, and any issues that were identified during testing;
- Audit documentation: Audit documentation includes reports, notes, and other materials related to the security audit of the smart contract. This type of documentation is critical in ensuring that the smart contract is secure and free from vulnerabilities.

These types of documentation are essential for smart contract development and maintenance. They help ensure that the contract is properly designed, implemented, and tested, and they provide a reference for developers who need to modify or maintain the contract in the future.

Remarks about Universal Address Audit documentation

The `Daimo Pay` team provided a `Notion` documentation detailing the technical details of the Universal Address Protocol smart contracts. Additionally, the team provided a comprehensive project walkthrough in the kick-off call with a detailed explanation of the intended functionalities. Moreover, the team addressed all questions and concerns the Nethermind Security team raised, providing valuable insights and a comprehensive understanding of the project's technical aspects.

8 Test Suite Evaluation

8.1 Compilation Output

```
user@machine contract % make full
make clean
rm -rf out lib
make build
forge install
Updating dependencies in /Users/kalzak/dev/nm/gauss/audits/0585/report/pay-audit/packages/contract/lib
forge build
[] Compiling...
[] Compiling 124 files with Solc 0.8.26
[] Solc 0.8.26 finished in 28.70s
Compiler run successful!
```

8.2 Tests Output

8.2.1 Foundry Tests Output

```
forge test --match-path "test/*"
No files changed, compilation skipped

Ran 4 tests for test/SharedConfig.t.sol:SharedConfigTest
[PASS] testNonOwnerCannotSetAddr() (gas: 16185)
[PASS] testOwnerCanSetAddr() (gas: 40607)
[PASS] testPauseFlag() (gas: 30621)
[PASS] testWhitelistedStableToggle() (gas: 44296)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 341.04µs (306.42µs CPU time)

Ran 1 test for test/PayBalanceReader.t.sol:PayBalanceReaderTest
[PASS] testBalanceReader() (gas: 2472726)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 517.08µs (261.46µs CPU time)

Ran 19 tests for test/Relayer.t.sol:RelayerTest
[PASS] testCheckSwapAndTipHash() (gas: 5700)
[PASS] testExcessOutputKeptByContract() (gas: 174369)
[PASS] testOnlyAdminCanGrantRelayerEOARole() (gas: 25391)
[PASS] testOnlyAdminCanWithdrawAmount() (gas: 81707)
[PASS] testOnlyAdminCanWithdrawBalance() (gas: 62774)
[PASS] testOnlyRelayerRoleCanClaimAndKeep() (gas: 37518)
[PASS] testOnlyRelayerRoleCanFastFinish() (gas: 49373)
[PASS] testOnlyRelayerRoleCanStartIntent() (gas: 40271)
[PASS] testOverPaymentRefunded() (gas: 196582)
[PASS] testSwapAndTipAuth() (gas: 38311)
[PASS] testSwapAndTipEmptyCall() (gas: 104701)
[PASS] testSwapAndTipPostSwap() (gas: 184676)
[PASS] testSwapAndTipPostSwapExcessiveTip() (gas: 156043)
[PASS] testSwapAndTipPreSwap() (gas: 181384)
[PASS] testSwapAndTipPreSwapExcessiveTip() (gas: 82008)
[PASS] testWithdrawAmountERC20() (gas: 71133)
[PASS] testWithdrawAmountNative() (gas: 44955)
[PASS] testWithdrawBalance() (gas: 53860)
[PASS] testWithdrawBalanceNative() (gas: 45614)
Suite result: ok. 19 passed; 0 failed; 0 skipped; finished in 671.96µs (1.46ms CPU time)

Ran 31 tests for test/UniversalAddress.t.sol:UniversalAddressTest
[PASS] testClaimIntent_NoFastFinish() (gas: 706419)
[PASS] testClaimIntent_SwapDeficitPull() (gas: 1431771)
[PASS] testClaimIntent_SwapEqual() (gas: 1385825)
[PASS] testClaimIntent_Underflow_Reverts() (gas: 638455)
[PASS] testClaimIntent_WithFastFinish() (gas: 734308)
[PASS] testClaimIntent_toTokenFewerDecimals() (gas: 1353903)
[PASS] testFastFinishIntent() (gas: 453203)
[PASS] testFastFinishIntent_Duplicate_Reverts() (gas: 465364)
[PASS] testFastFinishIntent_ViaRelayer() (gas: 3101929)
[PASS] testFastFinishIntent_WrongChain_Reverts() (gas: 340055)
```

```
[PASS] testFinishVaultIntent() (gas: 432740)
[PASS] testGlobalPause_Reverts() (gas: 104883)
[PASS] testRefundIntent_SourceChain_Succeeds() (gas: 1004359)
[PASS] testRefundIntent_WhitelistedToken_Reverts() (gas: 109347)
[PASS] testSameChainIntent_NoSwap_FullBalance() (gas: 295557)
[PASS] testSameChainIntent_NonWhitelistedToken_Reverts() (gas: 725290)
[PASS] testSameChainIntent_OutputLessThanToAmount_Reverts() (gas: 267898)
[PASS] testSameChainIntent_PaymentTokenFewerDecimals() (gas: 998992)
[PASS] testSameChainIntent_ToAmountLessThanMin_Reverts() (gas: 112712)
[PASS] testStartIntent() (gas: 331646)
[PASS] testStartIntent_BelowMinimum_Reverts() (gas: 112271)
[PASS] testStartIntent_DeficitPull() (gas: 354271)
[PASS] testStartIntent_InsufficientOutput_Reverts() (gas: 982501)
[PASS] testStartIntent_NonWhitelistedToken_Reverts() (gas: 724692)
[PASS] testStartIntent_OnDestinationChain_Reverts() (gas: 78391)
[PASS] testStartIntent_PaymentTokenFewerDecimals() (gas: 1035226)
[PASS] testStartIntent_PaymentTokenMoreDecimals() (gas: 1035180)
[PASS] testStartIntent_ReentrancyBlocked() (gas: 1096329)
[PASS] testStartIntent_SurplusRefund() (gas: 352668)
[PASS] testStartIntent_SwapPath() (gas: 1018138)
[PASS] testUniversalAddress_Reinitialise_Reverts() (gas: 160089)
Suite result: ok. 31 passed; 0 failed; 0 skipped; finished in 1.85ms (6.29ms CPU time)

Ran 30 tests for test/DaimoPay.t.sol:DaimoPayTest
[PASS] testAcrossFeeCalculation() (gas: 26170)
[PASS] testAcrossFromAndToTokenMismatch() (gas: 332581)
[PASS] testCCTPFromAndToTokenMismatch() (gas: 318640)
[PASS] testClaimWithFinalCall() (gas: 503994)
[PASS] testClaimWithInsufficientBalance() (gas: 297393)
[PASS] testClaimWithoutFastFinish() (gas: 305641)
[PASS] testCrossChainRefundAfterStart() (gas: 682604)
[PASS] testExecutorFinalCall() (gas: 196554)
[PASS] testExecutorFinalCallRefundExtra() (gas: 239640)
[PASS] testExecutorFinalCallRefundFailed() (gas: 217012)
[PASS] testExecutorOnlyCallableByEscrow() (gas: 27829)
[PASS] testExecutorRefundsSurplusTokens() (gas: 85937)
[PASS] testExpiredIntentCantClaim() (gas: 54047)
[PASS] testExpiredIntentCantFastFinish() (gas: 80539)
[PASS] testExpiredIntentCantStart() (gas: 92548)
[PASS] testExpiredIntentDestRefund() (gas: 418885)
[PASS] testExpiredIntentSourceRefund() (gas: 419591)
[PASS] testFastFinishWithPaidCall() (gas: 333920)
[PASS] testGetIntentAddr() (gas: 54391)
[PASS] testMaliciousIntentDrain() (gas: 327491)
[PASS] testMaliciousStartIntentOnDest() (gas: 353927)
[PASS] testSameChainRefundAfterClaim() (gas: 512528)
[PASS] testSimpleAcrossStart() (gas: 518184)
[PASS] testSimpleAxelarStart() (gas: 529288)
[PASS] testSimpleCCTPStart() (gas: 489896)
[PASS] testSimpleCCTPV2Start() (gas: 498969)
[PASS] testSimpleFastFinish() (gas: 175316)
[PASS] testSimpleFastFinishWithLeftover() (gas: 171652)
[PASS] testSimpleLPClaim() (gas: 358756)
[PASS] testSimpleSameChainStart() (gas: 316762)
Suite result: ok. 30 passed; 0 failed; 0 skipped; finished in 1.95ms (5.26ms CPU time)

Ran 5 test suites in 79.72ms (5.34ms CPU time): 85 tests passed, 0 failed, 0 skipped (85 total tests)
```

9 About Nethermind

Nethermind is a Blockchain Research and Software Engineering company. Our work touches every part of the web3 ecosystem - from layer 1 and layer 2 engineering, cryptography research, and security to application-layer protocol development. We offer strategic support to our institutional and enterprise partners across the blockchain, digital assets, and DeFi sectors, guiding them through all stages of the research and development process, from initial concepts to successful implementation.

We offer security audits of projects built on EVM-compatible chains and Starknet. We are active builders of the Starknet ecosystem, delivering a node implementation, a block explorer, a Solidity-to-Cairo transpiler, and formal verification tooling. Nethermind also provides strategic support to our institutional and enterprise partners in blockchain, digital assets, and decentralized finance (DeFi). In the next paragraphs, we introduce the company in more detail.

Blockchain Security: At Nethermind, we believe security is vital to the health and longevity of the entire Web3 ecosystem. We provide security services related to Smart Contract Audits, Formal Verification, and Real-Time Monitoring. Our Security Team comprises blockchain security experts in each field, often collaborating to produce comprehensive and robust security solutions. The team has a strong academic background, can apply state-of-the-art techniques, and is experienced in analyzing cutting-edge Solidity and Cairo smart contracts, such as ArgentX and StarkGate (the bridge connecting Ethereum and StarkNet). Most team members hold a Ph.D. degree and actively participate in the research community, accounting for 240+ articles published and 1,450+ citations in Google Scholar. The security team adopts customer-oriented and interactive processes where clients are involved in all stages of the work.

Blockchain Core Development: Our core engineering team, consisting of over 20 developers, maintains, improves, and upgrades our flagship product - the Nethermind Ethereum Execution Client. The client has been successfully operating for several years, supporting both the Ethereum Mainnet and its testnets, and now accounts for nearly a quarter of all synced Mainnet nodes. Our unwavering commitment to Ethereum's growth and stability extends to sidechains and layer 2 solutions. Notably, we were the sole execution layer client to facilitate Gnosis Chain's Merge, transitioning from Aura to Proof of Stake (PoS), and we are actively developing a full-node client to bolster Starknet's decentralization efforts. Our core team equips partners with tools for seamless node set-up, using generated docker-compose scripts tailored to their chosen execution client and preferred configurations for various network types.

DevOps and Infrastructure Management: Our infrastructure team ensures our partners' systems operate securely, reliably, and efficiently. We provide infrastructure design, deployment, monitoring, maintenance, and troubleshooting support, allowing you to focus on your core business operations. Boasting extensive expertise in Blockchain as a Service, private blockchain implementations, and node management, our infrastructure and DevOps engineers are proficient with major cloud solution providers and can host applications in-house or on clients' premises. Our global in-house SRE teams offer 24/7 monitoring and alerts for both infrastructure and application levels. We manage over 5,000 public and private validators and maintain nodes on major public blockchains such as Polygon, Gnosis, Solana, Cosmos, Near, Avalanche, Polkadot, Aptos, and StarkWare L2. Sedge is an open-source tool developed by our infrastructure experts, designed to simplify the complex process of setting up a proof-of-stake (PoS) network or chain validator. Sedge generates docker-compose scripts for the entire validator set-up based on the chosen client, making the process easier and quicker while following best practices to avoid downtime and being slashed.

Cryptography Research: At Nethermind, our cryptography Research team conducts cutting-edge internal research and collaborates closely with external partners on cryptographic protocols, consensus design, succinct arguments and folding schemes, elliptic curve-based STARK protocols, post-quantum security and zero-knowledge proofs (ZKPs). Our research has led to influential contributions, including Zinc (Crypto '25), Mova, FLI (Asiacrypt '24), and foundational results in Fiat-Shamir security and STARK proof batching. Complementing this theoretical work, our engineering expertise is demonstrated through implementations such as the Latticefold aggregation scheme, the Labrador proof system, zkvm-benchmarks, and Plonk Verifier in Cairo. This combined strength in theory and engineering enables us to deliver cutting-edge cryptographic solutions to partners and clients.

Smart Contract Development & DeFi Research: Our smart contract development and DeFi research team comprises 40+ world-class engineers who collaborate closely with partners to identify needs and work on value-adding projects. The team specializes in Solidity and Cairo development, architecture design, and DeFi solutions, including DEXs, AMMs, structured products, derivatives, and money market protocols, as well as ERC20, 721, and 1155 token design. Our research and data analytics focuses on three key areas: technical due diligence, market research, and DeFi research. Utilizing a data-driven approach, we offer in-depth insights and outlooks on various industry themes.

Our suite of L2 tooling: Warp is Starknet's approach to EVM compatibility. It allows developers to take their Solidity smart contracts and transpile them to Cairo, Starknet's smart contract language. In the short time since its inception, the project has accomplished many achievements, including successfully transpiling Uniswap v3 onto Starknet using Warp.

- **Voyager** is a user-friendly Starknet block explorer that offers comprehensive insights into the Starknet network. With its intuitive interface and powerful features, Voyager allows users to easily search for and examine transactions, addresses, and contract details. As an essential tool for navigating the Starknet ecosystem, Voyager is the go-to solution for users seeking in-depth information and analysis;
- **Horus** is an open-source formal verification tool for StarkNet smart contracts. It simplifies the process of formally verifying Starknet smart contracts, allowing developers to express various assertions about the behavior of their code using a simple assertion language;
- **Juno** is a full-node client implementation for Starknet, drawing on the expertise gained from developing the Nethermind Client. Written in Golang and open-sourced from the outset, Juno verifies the validity of the data received from Starknet by comparing it to proofs retrieved from Ethereum, thus maintaining the integrity and security of the entire ecosystem.

Learn more about us at nethermind.io.

General Advisory to Clients

As auditors, we recommend that any changes or updates made to the audited codebase undergo a re-audit or security review to address potential vulnerabilities or risks introduced by the modifications. By conducting a re-audit or security review of the modified codebase, you can significantly enhance the overall security of your system and reduce the likelihood of exploitation. However, we do not possess the authority or right to impose obligations or restrictions on our clients regarding codebase updates, modifications, or subsequent audits. Accordingly, the decision to seek a re-audit or security review lies solely with you.

Disclaimer

This report is based on the scope of materials and documentation provided by you to Nethermind in order that Nethermind could conduct the security review outlined in **1. Executive Summary** and **2. Audited Files**. The results set out in this report may not be complete nor inclusive of all vulnerabilities. Nethermind has provided the review and this report on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. Blockchain technology remains under development and is subject to unknown risks and flaws. The review does not extend to the compiler layer, or any other areas beyond the programming language, or other programming aspects that could present security risks. This report does not indicate the endorsement of any particular project or team, nor guarantee its security. No third party should rely on this report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.